

2 Time, causality and global state

Contents

- 2.1 Introduction
- 2.2 Physical time
 - 2.2.1 External synchronization
 - 2.2.2 Internal synchronization
 - 2.2.3 Compensation of deviations
 - 2.2.4 Examples
- 2.3 Logical time and ordering of events
 - 2.3.1 Event model
 - 2.3.2 Lamport's logical clocks
 - 2.3.3 Vector clocks
- 2.4 Global state and consistency
 - 2.4.1 System model
 - 2.4.2 Determination of consistent global states
 - 2.4.3 Algorithm of Chandy and Lamport
- 2.5 Exercises

2.1 Introduction

There are problems that, being trivial in centralized systems, are difficult to solve in distributed systems. A first example of these problems is to determine the **causality** relationship between events. The propagation delays of messages in a distributed system introduce a difficulty in establishing the cause-effect relationship of two events. This also occurs in other physical systems. Thus, for example, we can simultaneously perceive the figure drawn by fireworks and the sound of the cartridge's launch, which may lead us to think that the sound has been produced as a consequence of the figure's deployment. In this case, with a basic knowledge of physics, we could compensate the transmission times of the visual and audio signals by knowing the distances and the transmission speeds of light and sound through the air, and thus recompose the causality. In the world of computer networks, however, the situation is more complicated, since communication delays generally depend on variable parameters, such as network traffic or node load.

A second problem is related to the difficulty of providing to each node a **global view** of the state of the system, objective and **consistent** with that of other nodes. Since knowing the global state, distributed among the nodes, entails a cost derived from the need to communicate partial states, each node has a **subjective view** of the global state, sometimes inconsistent with that of other nodes. This problem is particularly sensitive to the scale of the system. For example, it is practically impossible to establish with absolute precision the amount of deposits in all the bank accounts of all the banks in the world at a given time.

An accurate mechanism of time measurement could help solving the problems of causality and the observation of the global state. In centralized and multiprocessor systems, which have a single clock, it is easy to determine the sequence of actions that make the state of the system evolve, since short distances allow a synchronous operation between the elements of the system. The single time reference provides a basis for establishing a total order among the system events (concurrent events can be ordered taking into account the process identifiers). In a distributed system one could think of dedicated lines to transmit the signal of a single clock, but this is not generally practical, for economic reasons, or it is totally unfeasible, as happens on the Internet.

It is necessary, therefore, to manage time in a distributed manner. Each node has its own clock, which provides a **local physical time**. If the clocks had absolute precision, and they were synchronized in their initialization (non-

trivial matter), the problem would be solved. In general, again for reasons of cost, the clocks are imprecise, so it is necessary to periodically synchronize the clocks of the nodes (that is, adjust them to a **reference physical time**).

Even when it is not strictly necessary to maintain absolute time references, the consistent management of the global state requires at least **ordering the events** produced by the nodes of the system, so that causality relationships can be determined. For this algorithms are used that provide a **logical time**. Frequently these algorithms use timestamps, generated by conveniently synchronized local clocks, associated with the events that are communicated by the system processes.

A logical time model will allow us to express the causality between events and introduce a **consistent global state** definition. From here, it is possible to design algorithms to determine global states, identifiable from each node of the system, which allows, for example, detecting deadlocks or implementing a distributed *debugger*.

2.2 Physical time

The clocks that are currently installed in computers or in any other device are based on quartz oscillators whose accuracy is not absolute, because the frequency of oscillation depends on several factors, some of them constants, which in turn depend on the characteristics of the oscillator itself, and some other variables, such as the ambient temperature. **Atomic clocks** can guarantee an almost absolute precision¹, but nowadays it is not practical to install them in current computers and devices.

From the signals generated by the oscillator of a local node in a distributed system, a sequence of instants of time is recorded, which is what determines the local physical time for that node, and which will sometimes have to be adjusted according to a reliable time reference.

It is necessary to introduce here a definition of time measurement. Time is a physical magnitude that is measured in seconds. But more than one definition of a second is possible:

¹ To know the state of this technology it can be consulted: W.J.H. Andrewes, *Medición actual del tiempo*, Investigación y Ciencia, Nov 2002. The NIST-F1, which is used as the main standard for measuring time in the United States, achieves a deviation of less than one second in 60 million years (<http://tf.nist.gov/timefreq/cesium/fountain.htm>).

Solar second. It is the measure of **astronomical time** and is defined as 1/86,400 of the period of rotation of the Earth. The problem is that the Earth does not rotate at a constant speed, but slowly loses speed, so the solar second can hardly be understood as an absolute reference for measuring time, although it is perfectly valid for situations of daily life.

Atomic second. It is defined as 9,192,631,770 transition periods in a Cesium-133 atom. This reference is constant² and was adopted in 1967 as the standard unit of what is known as **International Atomic Time (IAT)**. Atomic clocks, which use the Cesium atom as an oscillator, measure this time.

Due to the loss of speed in the terrestrial rotation, a drift accumulates between IAT and astronomical time (at present it is approximately $3 \cdot 10^{-8}$). As for everyday life on Earth it is more practical, at least in the long term, the astronomical time, a **Coordinated Universal Time (UTC)** has been defined, which is measured with atomic seconds and is updated by inserting a second from time to time³ to adjust it to astronomical time.

We will define below some concepts about clocks and their synchronization.

Reference physical time. It is the time supplied by a reliable source, to which we intend to adjust a local clock.

Resolution. Period between two updates of the local time register. It determines the ability of the system to totally order events over time. The resolution must be less than the minimum time interval between two consecutive events occurring in a node.

Deviation (*offset, skew, θ*). Difference between the local time recorded at an instant $\tau(t)$ and the reference physical time at that instant (normally t).

$$\theta(t) = \tau(t) - t$$

Drift (δ). Deviation per time unit of time (what advances or delays the clock).

$$\delta(t) = \frac{d\theta(t)}{dt} = \frac{d\tau(t)}{dt} - 1$$

² It is constant on Earth and only for a constant altitude, since time is affected by gravity, according to Einstein's Special Theory of Relativity.

³ The average insertion period so far has been 18 months.

Precision (*accuracy*, d). Maximum deviation that can be guaranteed when adjusting a clock.

Synchronization. Procedure by which the value of a clock is adjusted to a reference physical time with a pre-established precision. It is possible to use a reliable external source, such as UTC, as the time reference to synchronize a clock (**external synchronization**). For many applications, however, the important thing is not the adjustment to an absolute reference time, but the adjustment between the local times of the distributed system (**internal synchronization**), which allows ordering events reliably.

The purpose of the synchronization is to guarantee for each clock at all time instants a bound on the deviation with respect to the reference time (in internal synchronization, with respect to any other clock in the system). For the synchronization, depending on the deviation, the known drift and the precision of the adjustment, two parameters must be established:

Synchronization period. Maximum interval between a synchronization and the next one, that guarantees the bound established for the deviation.

Adjustment period. Interval during which the clock accelerates or decelerates to compensate a negative or positive deviation, respectively. The adjustment period must be less than the synchronization period.

Finally, we will consider that the system has a hardware clock and a software clock, which is maintained by the operating system. In general, synchronization operations are performed on the software clock.

2.2.1 External synchronization

The external synchronization of a clock is established in relation to a reference physical time coming from a reliable external source, such as UTC, so that the local clock expresses the universal time as closely as possible. The objective is to bound the deviation of the local clock with respect to the reference time. This is useful for global Internet applications or when the system simply intends to provide an accurate time service.

UTC time references can be obtained today from different sources, basically:

Land stations. Radio and television stations (the radio/TV receivers of today are prepared to receive them) and mobile telephony. Accuracy is affected by atmospheric conditions and distance to the station.

Satellites. Geostationary systems and geolocation systems. Although the former have the advantage of occupying a fixed point according to the terrestrial reference, in recent years the use of satellite fleets for geolocation has been generalized, particularly the GPS system of the US Army, open for civil uses (with restrictions). In these systems the distances are calculated according to the time it takes the electromagnetic signals to propagate, so the satellites are equipped with very well synchronized atomic clocks. GPS is not the only one, Russia, China, Japan and India have their own systems, and Europe is deploying its own, Galileo, which, being specifically for civilian use must provide more precision than GPS.

Internet. Subject to very variable delays. It is an alternative when the device does not have any type of receiver. In general, when using Internet synchronization, it is combined with internal synchronization mechanisms, such as NTP, which we will discuss later.

The accuracies of commercial receivers depend on the source used, as shown in Table 2.1.

Land stations	0,1—10 ms
GPS	0,1 μ s
Geostationary satellites	0,1 ms

Table 2.1. Typical accuracy of comercial UTC receivers.

2.2.2 Internal synchronization

For most applications, it is more important to keep the local clocks of the distributed system well synchronized with each other than to achieve high precision in external synchronization. Although each node might try to synchronize externally on its own, it is almost always more practical to maintain good internal synchronization between the nodes. The objective of the internal synchronization is to bound the deviations θ_{ij} between any pair of nodes i, j , which, assuming limited drifts in the local clocks, implies guaranteeing a certain precision in the adjustment and fixing a period of adjustment. There are several forms of internal synchronization, which are basically summarized in two types of algorithms: **centralized**, which use a specific server that provides time references, and **distributed**, which are usually statistical.

2.2.2.1 Centralized algorithms

Clients make requests to a time server that is supposed to provide reliable references⁴. A client⁵ C sends an m_{req} message to the time server, S , requesting the current time reference. The server replies by sending the time reference in an m_t message. The transmission of a message has a delay that has two components, $min+v$. min is the minimum time needed to send a message between two nodes of the system. It is constant and can be determined from the characteristics of the network. v is variable and depends mainly on the system load and network congestion. In general, it is not possible to establish an upper bound, although its distribution can be obtained empirically.

An example of a centralized algorithm is **Cristian's algorithm**. The precision of the adjustment of Cristian's algorithm is calculated in the following way:

Let $t(m_t)$ be the time reference returned by the server in the message m_t , and D be the interval between the sending of m_{req} and the reception of m_t , measured by the client⁶ (in t_1). Assuming that S assigns $t(m_t)$ at the (local) time instant τ , C will have sent m_{req} at the instant $\tau-(min+ds)$, and will have received m_t at the instant $\tau+(min+dr)$, where $D=2min+dr+ds$. Since min determines the minimum time to transmit a message, $t(m_t)$ will be in the range of values $dr+ds=D-2min$. Setting τ at the midpoint of the interval, we will obtain an accuracy $d=D/2-min$. Figure 2.1 tries to illustrate this calculation. It only remains to try D to be as small as possible, which is achieved by making several requests and using the minimum D for the adjustment. The more precision you want, the more requests you have to make, and good results are not guaranteed (for example, if the system is heavily loaded, very large values can be obtained for D).

To adjust the local clock, we must calculate the current deviation of the local clock⁷, θ .

⁴ Not necessarily UTC. What is important is to keep the clocks of the distributed system adjusted between them.

⁵ It can be understood as a client both a process and a node. Normally in each node there will be a single process in charge of adjusting the local clock.

⁶ Note that θ will be measured by a clock (that of the client node) that will present a drift δ with respect to the server time, then θ will suffer an error $\delta \cdot \theta$. However, it can be reasonably assumed that the drift is a very small magnitude (typically 10^{-5} or 10^{-6}), and therefore this error is negligible.

⁷ Since θ depends on time, strictly speaking this formula does not express the deviation in t_1 , but in $t_1-D/2$. However, since the drift is a very small magnitude and the indeterminacy inherent in the calculation of the adjustment is much greater, this error is irrelevant in practice.

$$\theta(t_1) = \tau(t_1) - t(m_t) = t_1 - \frac{D}{2} - t(m_t)$$

A problem with Cristian's algorithm, inherent to centralized algorithms, is its low fault tolerance due to the time server dependency. Fault tolerance can be improved by replicating the time server, but this introduces the need to synchronize servers if you want to keep deviations between clients bounded.

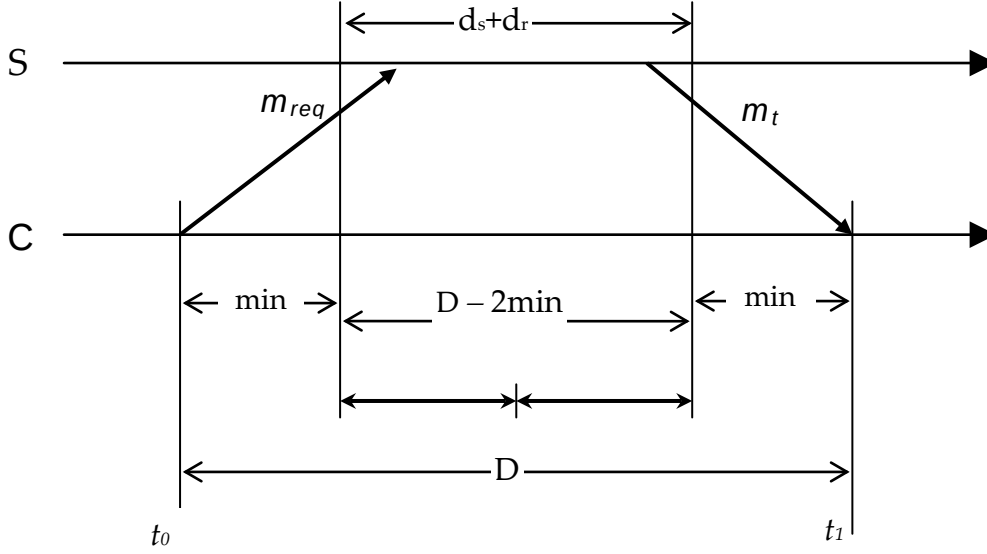


Figure 2.1. Calculation of the precision in the adjustment of Cristian's algorithm.

2.2.2.2 Distributed algorithms

While centralized algorithms try to adjust the local times with respect to a server or set of servers that provide the reference time, distributed algorithms are based on the statistical principle that the different deviations of the local times tend to cancel each other around an average time that can be taken as a reliable reference for internal synchronization.

In the **Berkeley algorithm**, a coordinator process in a node periodically requests the times of other nodes, estimating them according to communication delays (similar to Cristian's algorithm), and calculates an average time from the obtained measurements. In this statistical calculation, it is reasonable to exclude spurious responses (with very high deviations from the majority), which possibly correspond to nodes that have failed and do not have their clocks updated, and those that do not arrive within a certain period, which, in addition to not guaranteeing good precision, would delay the adjustment. The coordinator will send each node its deviation with respect to the calculated average, so that it adjusts its local time. Note that any process can be elected as

a coordinator at any given time, which makes this algorithm inherently distributed and fault-tolerant.

The average time and precision are estimated following the idea of Cristian's algorithm. To simplify, we will consider that we do not know the minimum delay of the communication (that is, $\min=0$ for every message between each pair of nodes). If the coordinator process, P_0 , sends the request message at a reference time t_0 , and receives N responses with non-spurious measures (including his), the average time T calculated at t_0 will be:

$$T(t_0) = \frac{\sum_i (t(m_i) - D_i/2)}{N}$$

so the deviation to communicate to a process P_i will be⁸:

$$\theta_i(t_0) = t(m_i) - \frac{D_i}{2} - T(t_0)$$

and the precision of the adjustment at i , $d_i=D_i/2$.

Another example of distributed algorithm is the symmetric method of NTP, which we will study later.

2.2.3 Compensation of deviations

Once a deviation in the local physical time has been detected, it is necessary to adjust it, a task that does not simply consist of adding or subtracting the deviation detected, since a fundamental property of time could be violated: its increasing monotonicity. Some applications, such as the UNIX *make* (see Exercise 4) are sensitive to this property.

The deviation compensation is applied to a software clock⁹. Two cases can occur for the adjustment:

- Negative deviation. The same amount can be added to the local physical time to compensate the deviation, which only produces a discontinuity in time (Figure 2.2a), without further consequences for the applications

⁸ Logically, the deviation is calculated and sent to all processes, including spurious ones.

⁹ Applications that require real time cannot rely on this clock. The *gettimeofday()* function accesses the software clock, while some systems provide functions to access the hardware clock, such as *gethrtime()*.

- Positive deviation. If the local physical time were subtracted, the monotonicity would be broken, causing situations of temporary inconsistency. Instead, the software clock is slowed down during a period of adjustment, as shown in Figure 2.2b. For applications, time simply runs slower during that period. This adjustment method can be generalized for deviations of both signs¹⁰.

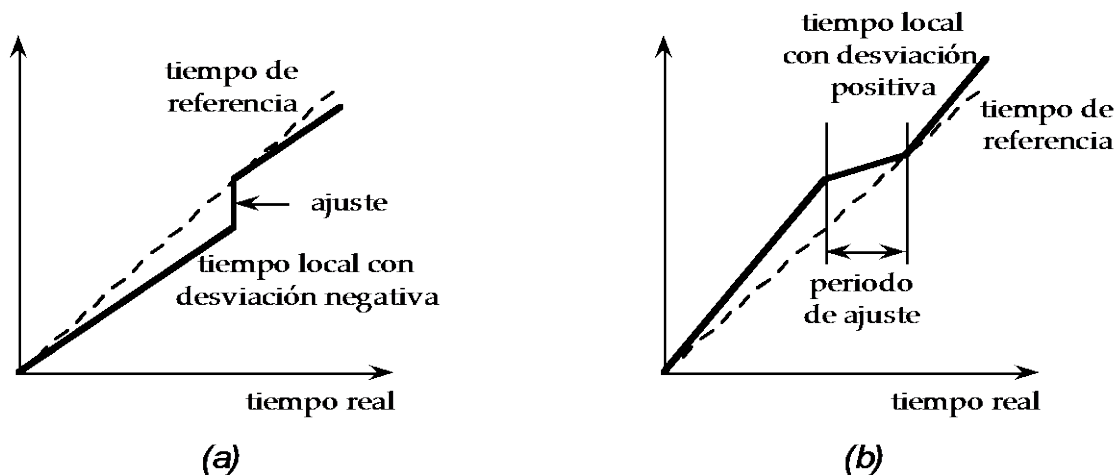


Figure 2.2. Compensation of the deviation (a) negative, (b) positive.

The duration of the adjustment period has two restrictions. On the one hand, the slope during the adjustment period should be sufficient for the resolution of the clock in that period to be less than the minimum time interval between two events occurring consecutively at the node. On the other hand, the adjustment period must be less than the synchronization period.

2.2.4 Examples

2.2.4.1 Distributed Time Server

An example of distributed algorithm is the one used in the DCE *Distributed Time Server* (DTS). Physical times are represented by intervals. There are two types of nodes, *time servers*, which maintain external time references (for example UTC), and clients, *time clerks*, local daemons that periodically synchronize their local clocks with remote time servers.

¹⁰ UNIX provides the system call *adjtime()* for this type of adjustment. Linux provides *adjtimex()*, which offers greater functionality.

Time clerks adjust their clocks from the intervals provided by the time servers, using the midpoint of the interval of greatest intersection as the midpoint of their new time interval, as represented by the example in Figure 2.3.

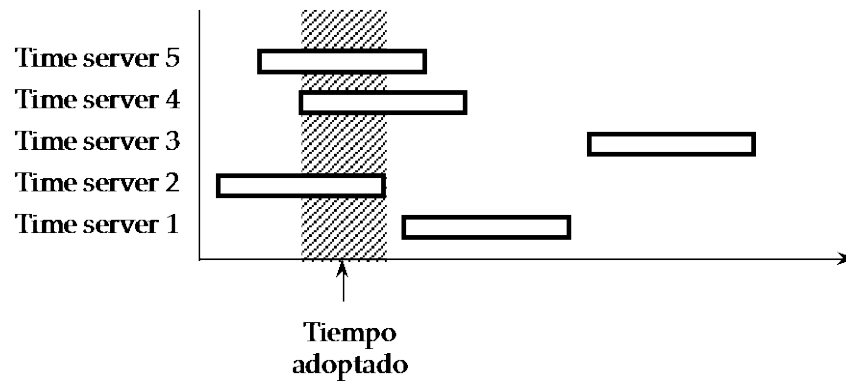


Figure 2.3. Time adjustment algorithm in Distributed Time Server of DCE.

2.2.4.2 Network Time Protocol

The *Network Time Protocol* (NTP)¹¹ has become standard on the Internet, a time service that covers the entire Internet network. The objective is to provide synchronization with UTC in a redundant manner. The system is structured in layers (*strata*) of time servers. The upper layer is composed of a set of *primary servers* that are supposed to maintain reliable UTC references. In the next layer, the *secondary servers* are kept synchronized with respect to the primary ones. The lower layers correspond to the clients, being able to extend the hierarchy indefinitely. In general, the accuracy of external synchronization (with respect to UTC) decreases as the number of network hops (*strata*) increases to the UTC source.

NTP provides several modes of operation. In local networks with adequate support and bounded communication times, it is possible to operate in *multicast* mode, where a server periodically sends the time reference to the nodes of the network. When this support is not available or a greater precision is required, the *procedure call* mode is more suitable, which allows a client to synchronize with a reference server, analogous to Cristian's algorithm. When a better internal synchronization is required, it can operate in *symmetric* mode. Two servers operating in symmetric mode exchange time information to adjust the clocks of both.

¹¹ <http://www.ntp.org>

In the procedure call and symmetric operation modes, NTP uses an algorithm similar to Cristian's to calculate the deviation and the precision of the adjustment. Minimum delays in transmission are not considered a priori known. However, the symmetric mode compensates communication delays through a double exchange of time information.

Another feature of NTP is the ability to reconfigure the logical structure of the server network based on the availability or accuracy of the servers. For example, if a stratum 1 server fails, it may fall to stratum 2.

More details about NTP and its synchronization algorithms can be found in [COU05 §10 and MIL91].

2.3 Logical time and ordering of events

From what has been seen in the previous section, it can be deduced that, due to the drift of the clocks and the difficulty in bounding the precision of the adjustment, physical time is not, in general, a reliable mechanism to temporarily order the events produced in different nodes of a distributed system, which can introduce distortions in the causality relationships between events. On the other hand, events of the same node can be ordered according to the moment of local physical time in which they occur, provided that the monotonicity of the time in the adjustment process is respected, as we have seen above. On the other hand, the important thing for most applications is not the time instant in which the events take place, but the relative order between them. This order can be defined in different ways and establishes a **logical time** to synchronize system events.

2.3.1 Event model

We will define a distributed system model with a set of processes¹² that communicate by message passing:

$$\text{send}(P_i, \text{message})$$
$$\text{receive}(P_j, \text{message})$$

¹² For the moment we will assume one process per node. As we will see, the model is still valid if several processes are considered per node.

P_i and P_j represent respectively the process that sends and receives the message. Each process generates a sequence of events, which can be of three types:

- Sending a message. When executing *send*.
- Receiving a message. When *receive* delivers the message to the application.
- Internal. Any other event that does not involve communication.

For events in the same node it is possible to establish a total order, for example based on the physical time in which the events occur. If the local time in which the event occurs, $T(x)$, is associated to each event x as a timestamp, all the events of the node will be **temporarily ordered** among them. The resolution of the clock must be such that it allows associating different timestamps to any two consecutive events. Several processes could produce simultaneous events¹³, and in that case the temporal relationship is solved by the process identifier. However, since what interests us is the order of events rather than the physical time instant in which they occur, a simple event counter serves as a logical clock.

We can establish relationships between pairs of events generated in the system. It is said that two events are **causally related** if they are events of the same process or if they correspond to the sending of a message by a process and the reception of the message by the destination process. It is a partial order relationship that we will call *happened before* ($\rightarrow$). More formally, we will denote $x \rightarrow y$ (x happened before y) if:

1. x and y are events of the same process and $T(x) < T(y)$.¹⁴
2. Given a message m , x is the event produced by *send*(m) and y is the event produced by *receive*(m).
3. There exists an event z such that $x \rightarrow z$ and $z \rightarrow y$ (transitive property).

Two events x and y for which the *happened before* relationship cannot be established are **concurrent** events ($x \parallel y$).

¹³ For example in a multiprocessor.

¹⁴ We assume that an event x occurs in a time $T(x)$, but this time is ideal and is not available to processes.

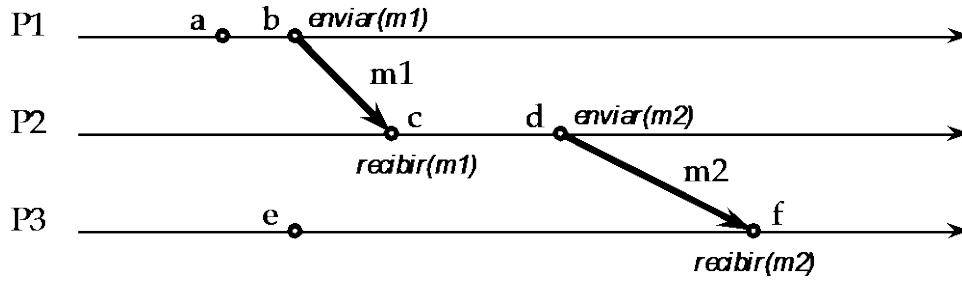


Figure 2.4. Chronogram with causally related events and concurrent events.

Figure 2.4 shows how these relationships are represented by a chronogram. Note that, for example, $a \rightarrow b$, $b \rightarrow c$, $a \rightarrow c$, $a \rightarrow f$, $a \parallel e$, $e \parallel d$.

2.3.2 Lamport's logical clocks

Lamport described an algorithm that allows to represent the *happened before* relationship in a distributed system. Each process has its own **local logical clock** that generates timestamps to associate with the events it produces. The local logical clock can be the local physical clock, but a simple asynchronous counter associated with each process is adequate.

Lamport's algorithm establishes how the local logical clock C_i of each process P_i is updated, and how the logical clock is associated as the timestamp of each event.

- Initially, $C_i = 0$, $\forall i$.
- Before executing a *send* event or an internal event at P_i :

$$C_i = C_i + 1$$

- When sending a message m , the sender, P_j , includes in the message the current value of its logical clock, C_m .
- Upon reception of the message at the destination process, P_i , the local clock C_i is updated as follows:

$$C_i = \max(C_i, C_m)$$

$$C_i = C_i + 1$$

It can be seen that the values of the marks associated with the events do not fully express the order relation between the events (see Exercise 5). In particular, $C_i(x) < C_j(y)$ does not imply $x \rightarrow y$ if x is an event of P_i and y is an event

of P_j for $i \neq j$. It can be established, however, that if two events have the same mark, then they are concurrent ($C_i(x)=C_j(y) \Rightarrow x \parallel y$). We will try to solve this problem in the following section.

2.3.3 Vector clocks

The causality or concurrence of any two events can be determined if information about the logical clocks of other processes is introduced in each process. Each process P_i has now a vector V_i of N entries, being N the number of processes, where $V_i[i]$ represents the local logical clock of P_i and $V_i[j]$, for all $j \neq i$, represents the last value of the logical clock of P_j known by P_i .

The algorithm to manage the vector clocks of each process is the following:

- Initially, $V_i[j]=0, \forall i, j$.
- Before executing a *send* event or an internal event at P_i :

$$V_i[i]=V_i[i]+1$$

- When sending a message m , the sender, P_j , includes in the message the current value of its vector clock, V_m . Upon reception of the message at the destination process, P_i , the local vector clock V_i is updated as follows:

$$\forall k: V_i[k]=\max(V_i[k], V_m[k])$$

$$V_i[i]=V_i[i]+1$$

With this algorithm, if x is an event of P_i and y is an event of P_j , $x \rightarrow y$ is true if and only if the following is true:

$$V_i(x) < V_j(y)$$

that is to say:

- $V_i[k](x) \leq V_j[k](y), \forall k$
- $\exists k \mid V_i[k](x) \neq V_j[k](y)$

The first condition indicates that P_j keeps updated information of the logical clock of P_i , something that will happen if and only if it has been transmitted in a message or P_j has updated it himself. The second condition establishes the non-reflexibility of the causality relation.

A more formal description can be found in [MUL93 §4].

2.4 Global state and consistency

While it is easy to define the sequence of states of an isolated process, if we consider the distributed system as a whole, defining the global state of the system is very complicated. This is because a process has a subjective view of the system, based on the messages it receives and its own perception of time. There is no *global observer* with an objective view of the state. Since it is not possible to keep local times perfectly synchronized, we cannot rely on timestamps to globally order local states. The global state of the distributed system must be inferred, therefore, from the individual states of the processes.

The determination of global states is useful for debugging errors, detecting deadlocks or determining the termination of a set of processes.

2.4.1 System model

We will consider a set of N processes P_i ($i = 1, 2, \dots, N$). We will define the **history of a process** P_i as the sequence of events that occur in P_i :

$$H_i = \langle e_i^0, e_i^1, e_i^2, \dots \rangle$$

We can consider a **prefix of the history** of a process until the execution of a given event, e_i^k :

$$H_i^k = \langle e_i^0, e_i^1, \dots, e_i^k \rangle$$

We will denote as s_i^k the **status of a process** P_i immediately after the event e_i^k occurs. s_i^0 is the initial state of process P_i .

At a certain time, the global state of the system could be defined as the collection of local states, $S = \{s_1, s_2, \dots, s_N\}$. However, this definition would include inconsistent global states. To define consistent global states we will introduce previously the concept of **cut** of the execution in the distributed system, C , as a set of prefixes of the histories of the processes:

$$C = H_1^{c_1} \cup H_2^{c_2} \cup \dots \cup H_N^{c_N}$$

where $H_i^{c_i}$ contains the events processed by P_i included in the cut. We will define as **cut border** the set of events $\{e_i^{c_i}, i = 1, 2, \dots, N\}$, where $e_i^{c_i}$ is the last event of $H_i^{c_i}$.

A cut C is **inconsistent** when there exist a pair of events $e_j \in C$ and $e_i \notin C$ such that $e_i \rightarrow e_j$ (Figure 2.5a).¹⁵ Figure 2.5b shows two examples of **consistent cuts**. In other words, the reception of a message only belongs to a consistent cut if the sending of the message also belongs to the cut.

Given the definition of state introduced above, we can define a **consistent global state** as that which corresponds to a consistent cut. A sequence of transitions of consistent global states in a distributed system determines a **consistent execution**.

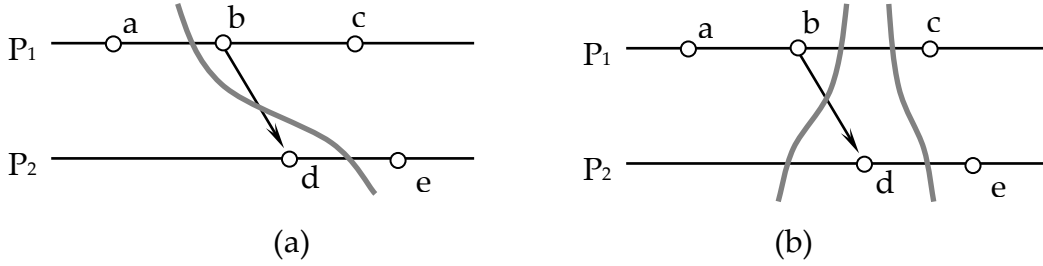


Figure 2.5. (a) Inconsistent cut. (b) Two consistent cuts.

Note that in the definition of the state of the system, in addition to the state of each process, determined by its history, we must consider the state of the communication channels between processes. We will denote by c_{ij} the state of the communication channel by which process P_j receives messages from process P_i . c_{ij} is constituted by the set of messages sent by P_i and not yet received by P_j in the cut.

2.4.2 Determination of consistent global states

Given a cut, the vector clocks can be used to determine whether it is consistent or not. Indeed, in the cut of Figure 2.5a, the vector of P_2 will have a timestamp in the event d on the clock of P_1 bigger than that of the vector of P_1 in a , since it has been updated in event b , which is out of the cut:

$$V_2[1](d) > V_1[1](a)$$

Therefore, a cut $C = H_1^{c_1} \cup H_2^{c_2} \cup \dots \cup H_N^{c_N}$ is consistent if and only if, for any pair of events e_i, e_j of the cut border:

$$V_i[i](e_i) \geq V_j[i](e_j)$$

¹⁵ Actually, it is enough checking the events of the cut border.

What can be interpreted intuitively is that, at the cut border, no process P_j has more updated information of the logical clock of a process P_i than the process P_i itself. In other words, if in a cut C a process P_j includes the event corresponding to a message received from P_i , while the event corresponding to the sending event of the message by P_i is not included in C , then C is inconsistent.

Determining the consistency of a state from the vector clocks is not possible from a particular node, but only from an external observer who had a global and simultaneous view of all the vectors. Determining a global state should be considered as the construction of a consistent cut based on the processes communicating their local states, for which the algorithm of Chandy and Lamport is used. This algorithm builds *snapshots* of the states of the processes, which also include the state of the channels through which they receive messages.

2.4.3 Algorithm of Chandy and Lamport

The snapshot algorithm, proposed by Chandy and Lamport, constructs a global state at the initiative of any process (initiator).

It is based on the following assumptions:

- There is a unidirectional FIFO communication channel between all pairs of sender-receiver processes.
- Neither the processes nor the channels fail during the execution of the algorithm.
- Any process can start the algorithm at any time.
- Processes can continue to send and receive *normal* messages (*application messages*) during the execution of the algorithm.

In addition to the process state itself, each process constructs the state of its reception channels. As we have already defined, the messages sent by process P_i and not yet received by process P_j constitute the state of the c_{ij} channel.

The algorithm uses a special *SNAPSHOT* message to synchronize the processes. We will assume that process P_1 is the initiator.

- 1 P_1 saves its state s_1 and broadcasts a *SNAPSHOT* message to the rest of processes. From now on, the application messages will be stored in the $c_{k1} \forall k$ channels.

- 2 When a process P_i receives a *SNAPSHOT* message (from process P_j , be it the initiator or another process), if P_i has not saved its state yet:
 - (a) P_i saves its state s_i .
 - (b) P_i saves the state of channel c_{ji} as empty. In the rest of channels $c_{ki} \forall k \neq j$, application messages will be stored from this moment.
 - (c) P_i forwards the *SNAPSHOT* message to the rest of processes (between the reception of the *SNAPSHOT* message and its forwarding P_i does not execute any other event).
- 3 When process P_i receives a *SNAPSHOT* message from a process P_k ($k \neq j$, being P_j the process from which P_i received the first *SNAPSHOT* message), and P_i has already saved its state, the state of channel c_{ki} will be complete and it is thus saved. When process P_i has received a *SNAPSHOT* message from all the processes, the construction of the global state at P_i is terminated.

Note that when the state of a process P_i is saved, the storage of messages in the process input channels is activated. Messages that process P_i receives from a process P_j that has not yet saved its state (that is, those that P_i receives before P_j 's *SNAPSHOT* message) will be stored in the c_{ji} channel, since these messages have been sent before the state saving of P_j (given the FIFO nature of the channels), and therefore the send event belongs to the cut, while the reception event at P_i is outside the cut. The channel will be saved upon reception of P_j 's *SNAPSHOT* message (step 3).

It can be checked that the algorithm terminates and builds a consistent global state (see for example [COU05]).

2.5 Exercises

1 To synchronize our local clock we are using a remote time service. We do not know any characteristics of this service in terms of communication delays. We just make a time request and measure the interval between sending our request and receiving the response through the *gettimeofday()* function.

- (a) If in a single measurement we obtain a value of the interval of 482 *ms*, what is the precision that we can estimate?
- (b) The difference between the time value returned by the server, $t(m)$, and our local clock at the time of reception of the response, t_l , is $t_l - t(m) = -2691$ *ms*. Calculate the deviation of the local clock at t_l .
- (c) If at the time of this measurement, 24 hours have passed since the last adjustment, calculate the local clock drift.
- (d) As the precision obtained in our measurement has not left us satisfied, we want to improve it for the next adjustment. Describe what must be modified in the measurement method to achieve it.

2 A daemon that manages the local physical time of a node in a network is periodically synchronized with a remote time service to adjust the local clock. To do this, it uses Cristian's algorithm, making several requests to the server in each synchronization operation, in order to achieve the best possible accuracy. For each request, the daemon records: the time elapsed between sending the request message and receiving the message with the server response, D , the local time in which the response message is received, t_l , and the remote time received in the reply message, $t(m_l)$. The resolution of the clocks is 1 μs . It is known, by the characteristics of the network, that the minimum time necessary to transmit a message is 1 *ms*, and the maximum time is not bounded.

In a synchronization operation, the daemon has made 4 requests, obtaining the results shown in the table. According to these results, calculate the best possible precision in the adjustment and the deviation of the local clock in t_l (estimated by the daemon according to the best precision).

Request	D (μ s)	t_i (s, μ s)	$t(m_i)$ (s, μ s)
1	8030	630123798,567328	630123798,458124
2	3420	630123803,595970	630123803,506117
3	25420	630123808,801872	630123808,710843
4	5289	630123813,890012	630123813,804975

3 Following the Berkeley algorithm, a coordinating node N1 periodically polls all the nodes of a system to calculate an average time and send each node the value of its corresponding deviation. In one of the polls N1 has received from each node the local time values indicated in the table. Likewise, for each node, it has recorded the interval D between sending the request message and receiving the message with the local time, which is also indicated in the table.

Node	T. local (s, ms)	D (ms)	d (question b)	θ (question c)
N1	800123799,958	0		
N2	800123803,506	20		
N3	800123808,710	31		
N4	800123800,797	16		
N5	800110525,386	42		

- To calculate the average time, will the coordinator exclude any of the nodes? Why?
- Taking into account that the minimum delay in the communication of a message is not known, calculate the precision d of the adjustment for each node (use the Table to write the values).
- Calculate the value of the adopted average time and the deviation values θ to be sent to each node. (Consider that the time request messages are broadcast simultaneously by N1 to the rest of nodes).

4 A distributed system uses a time synchronization algorithm that adjusts the local time immediately after detecting a deviation, adding or subtracting the value of the deviation. In this system we are developing an application in C on UNIX using the *make* utility. We have two source files, a.c and b.c. At 9h30:00 (local time of the node) we execute make and obtain the files a.o, b.o and a.out (the executable). We execute a.out and detect a fairly trivial error. At 9.30:30 we edit the file a.c and correct the error, which takes exactly 15 seconds. (While editing, the time synchronization algorithm has been executed and it has

detected an advance of 1 minute in our node, which has been corrected according to the exposed criteria.) Immediately we save the a.c file and execute make. We run a.out again and observe with surprise that the error persists. Explain the reason.

5 The chronogram of Figure 2.6 shows the relationships between events produced by processes that run on different nodes of a distributed system.

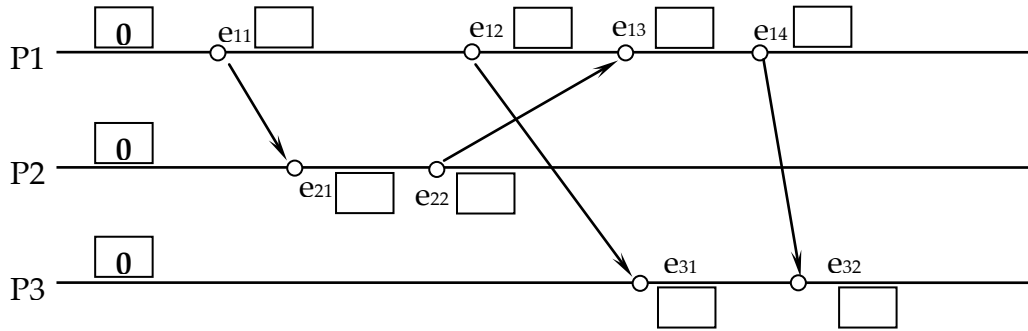


Figure 2.6

- Give three examples of pairs of concurrent events and three examples of causally related events.
- Assuming that we intend to order the events by applying the Lamport algorithm, indicate on the rectangles of the chronogram of the figure the values that the timestamps C_i of each process P_i take.
- Indicate on the chronogram of Figure 2.6 an example that does not fulfill the property $C(e_i) < C(e_j) \Rightarrow e_i \rightarrow e_j$.

6 From the chronogram of Figure 2.6, calculate the values of the vector clocks of each process that are associated with each event. Check for the pair of events of section (c) of the previous exercise that it is now possible to determine whether or not a causal relationship between them exists.

7 On the chronogram of Figure 2.6, draw a consistent cut and an inconsistent cut. For the consistent cut determine the state of the channels.

8 Given the processes of Figure 2.7, which produce message sending and receiving events,

- Determine the values of the clocks according to the algorithm of the vector clocks.

- (b) Assume that process P2 initiates the execution of the Chandy-Lamport algorithm to determine the global state of the system by sending SNAPSHOT messages (arrows with dashed lines in Figure 2.8). (b1) Draw on the chronogram a possible execution of the Chandy-Lamport algorithm. (b2) Draw the cut that represents the recorded global state. (b3) Describe that global state, indicating the state of each process and that of each of the six communication channels.

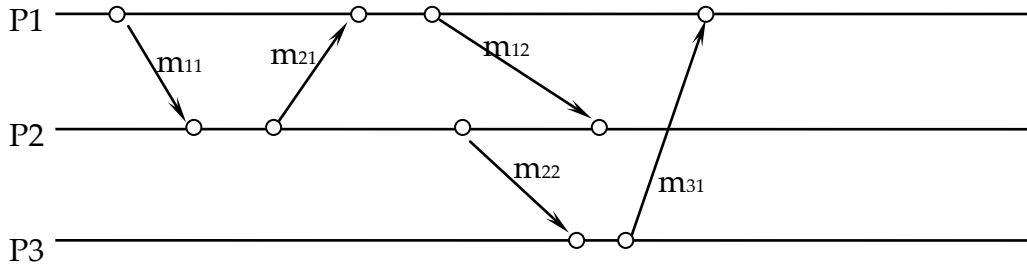


Figure 2.7

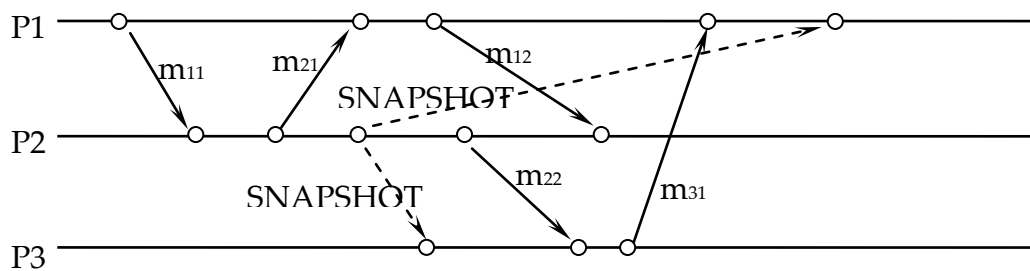


Figure 2.8

- (c) From the information provided by the vector clocks, demonstrate that the cut of the previous section is consistent.